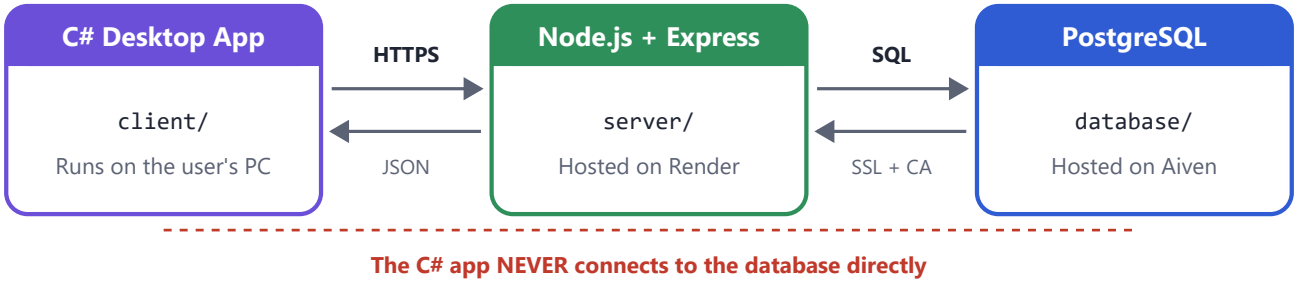
AGP INVENTORY SYSTEM · BEGINNER'S GUIDE

Building Your First Full-Stack App

C# desktop app · Node.js + Express API on Render · PostgreSQL on Aiven



Everything a beginner needs: what to install, how the repository is organised, how to run it, and step-by-step how-tos for writing a function, creating a page, opening a page, reading from the database and storing data. Every code example in this guide comes from the repository or was built and tested before being printed here.

Version 2.0 · November 2026 · github.com/Shaloh69/AGP-Inventory-System

Contents

1. Read This First . . . 3
2. The Big Picture . . . 4
3. Tools and Accounts . . . 6
4. The Repository . . . 7
5. Project Plan and Timeline . . . 8
6. Set Up the Database on Aiven . . . 9
7. Run Everything on Your PC . . . 11
8. How the Server Works . . . 13
9. Deploy the API on Render . . . 15
10. How the C# App Works . . . 16
11. How To: Write Your First Function . . . 18
12. How To: Create a Page . . . 21
13. How To: Open a Page and Pass Data . . . 24
14. How To: Call the Database (Read) . . . 26
15. How To: Store Data (Create, Update, Delete) . . . 29
16. How To: Add or Change a Table (Migrations) . . . 33
17. Tutorial: Build a Complete Feature (Suppliers) . . . 34
18. How To: Debug . . . 38
19. UI Libraries for C# . . . 39
20. Working as a Team with Git and GitHub . . . 41
21. Security Basics . . . 42
22. Next Step: Adding Login (bcrypt + JWT) . . . 43
23. Troubleshooting . . . 44
24. Final Checklist . . . 45
25. Glossary and Useful Links . . . 46

SECTION 1

Read This First

This guide is for people who have never built a full application before. The repository is a working starting template, so you learn by running, reading and changing real code.

What is already in the template

- A **C# desktop app** with a sidebar menu. Each menu item is a **page**: Home, First function, Products, and one page per UI library.
- An **Express API** with a complete **products** feature: list, search, add, edit, delete, with validation and clear error messages.
- A **PostgreSQL** table (`products`) created by a numbered migration, plus sample data.
- A one-click runner, `run-dev.bat`, that starts everything on your PC.

Your job is to add the rest of the inventory system (suppliers, categories, stock in/out, users, reports) by copying the products pattern. Section 17 walks you through one complete new feature, step by step.

How to use this guide

If you want to…	Read
understand how it all fits together	Sections 2, 4, 8 and 10
get it running on your PC	Sections 3, 6 and 7
put the API online	Section 9
learn by doing	Sections 11–17 (the How-To sections)
fix an error	Section 18 (Debug) and Section 23 (Troubleshooting)

The five golden rules

#	Rule	Why
1	The C# app never talks to the database directly.	If the database password were inside the .exe, anyone could extract it and delete your data.
2	Secrets live in `.env` / Render settings, never in code or Git.	Bots scan GitHub for passwords within minutes.
3	Always use SQL placeholders (`$1, $2`), never glue values into SQL.	Prevents SQL injection, the most common way databases get hacked.
4	Never store plain-text passwords. Hash them with bcrypt.	If the database leaks, users' real passwords stay safe.
5	Small steps, test often, commit often.	When something breaks you only have a little code to check.

SECTION 2

The Big Picture

Before changing any code, understand what each part does and how one click travels through the whole system.

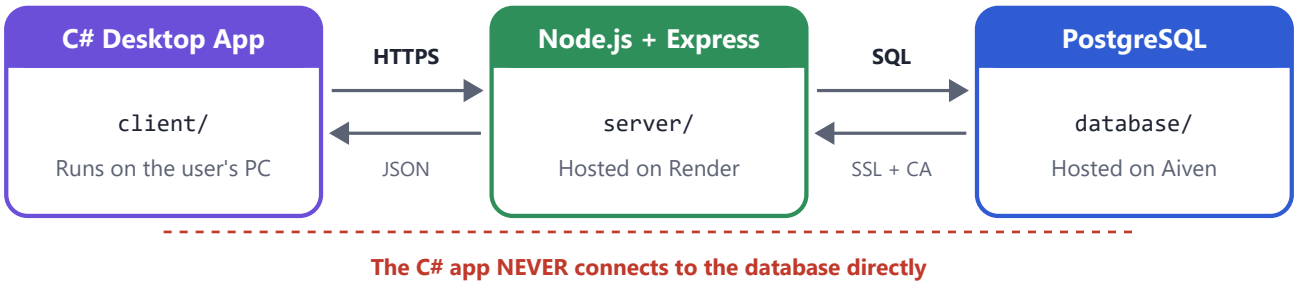


Figure 1 – The three layers and how they communicate

Layer	Technology	Its job	Where it runs
Frontend (client/)	C# · .NET 10 · WinForms + WPF pages	Shows pages, collects input, displays results. Contains no passwords and no SQL.	On each user's Windows PC
Backend (server/)	Node.js + Express	Receives HTTP requests, checks them, runs SQL, returns JSON.	Render (cloud)
Database (database/)	PostgreSQL	Stores data permanently in tables.	Aiven (cloud)

One click, step by step

Example: on the Add product page the user types *Claw Hammer*, 25, 349.00 and clicks **Save**.

1. `ProductFormPage.btnSave_Click` copies the form into a `Product` object and calls `ApiClient.CreateProductAsync(product)`.
2. ApiClient sends `POST /api/products` with the body `{"name":"Claw Hammer","quantity":25,"price":349}`.
3. Express matches the URL in `product.routes.js` and calls `controller.create`.
4. The controller checks the input with `validateProduct`, then calls the model.
5. The model runs `INSERT INTO products (name, quantity, price) VALUES ($1, $2, $3) RETURNING *`.
6. PostgreSQL saves the row. Express replies `201 Created` with the new product as JSON.
7. The app goes back to the Products page, which loads the list again, now with the new product.

Key words

Term	Meaning
API	A set of URLs your app can call to read or change data.
Endpoint	One URL + method, e.g. `GET /api/products`.
HTTP method	The action: `GET` read, `POST` create, `PUT` update, `DELETE` remove.
JSON	Text format for data: `{ "id": 1, "name": "Claw Hammer" }`.
Status code	The result: `200` OK, `201` created, `204` deleted, `400` bad input, `404` not found, `409` duplicate, `500` server error.
Page	A UserControl shown inside the main window when you click a menu item.
Migration	A numbered SQL file that creates or changes tables.

SECTION 3

Tools and Accounts

Every team member installs the tools. One person creates the Render and Aiven services and invites the others.

Tool	Used for	Get it from	Check it works
Git	Version control, sharing code	git-scm.com	`git --version`
Node.js (LTS, 20 or newer)	Running the Express server	nodejs.org	`node -v`
Visual Studio 2022 or newer (Community)	Writing the C# app. Tick **.NET desktop development**.	visualstudio.microsoft.com	Opens client/AppClient.slnx
.NET 10 SDK	Building the C# app (usually installed with Visual Studio)	dotnet.microsoft.com	`dotnet --version`
VS Code	Editing the Node.js server	code.visualstudio.com	Opens the server folder
DBeaver or pgAdmin (optional)	Looking inside the database	dbeaver.io / pgadmin.org	Connects to Aiven
Postman, Bruno or Thunder Client	Testing API endpoints without the app	postman.com / usebruno.com	Can send a GET request

Account	Why	Notes
GitHub	Stores the repository; Render deploys from it.	Accept the collaborator invite for Shaloh69/AGP-Inventory-System.
Render (render.com)	Hosts the Express API.	Sign up with GitHub. The free plan is enough.
Aiven (aiven.io)	Hosts PostgreSQL.	Free PostgreSQL plan. Invite teammates to the project.

CHECKPOINT Run `run-dev.bat check` from the repository root. Every tool should show **[ok]**.

SECTION 4

The Repository

Where everything lives, and which file to open for each kind of change.

Folder structure

```
AGP-Inventory-System/
├── run-dev.bat / run-dev.ps1     run everything locally
├── client/                       C# app (open AppClient.slnx)
│   ├── AppClient/
│   │   ├── Program.cs            starts the app (--page Name opens a page)
│   │   ├── Forms/MainForm.cs     main window: sidebar menu + page area
│   │   ├── Pages/                HomePage, FirstFunctionPage, ProductsPage, ProductFormPage
│   │   │   └── Libraries/        MaterialSkin, ReaLTaiizor, Krypton demo pages
│   │   ├── Models/Product.cs     matches the JSON from the API
│   │   ├── Services/ApiClient.cs ALL calls to the server
│   │   └── Helpers/              AppConfig, InventoryMath, Navigator, Theme
│   ├── AppClient.WpfPages/       MaterialDesign, WPF UI, MahApps, HandyControl pages
│   └── capture-pages.ps1         re-takes the screenshots in docs/
├── server/                       Express API
│   ├── .env.example              copy to .env
│   ├── certs/                    Aiven ca.pem (never committed)
│   └── src/
│       ├── index.js, app.js      start-up and middleware
│       ├── config/db.js          PostgreSQL connection
│       ├── routes/  controllers/  models/     the products feature
│       ├── middleware/           errorHandler, notFound
│       ├── utils/                validateProduct
│       └── scripts/              migrate.js, seed.js
├── database/
│   ├── migrations/               001_create_products_table.sql ...
│   └── seeds/                    001_sample_products.sql
└── docs/                         this guide, start page, screenshots
```

Where to start coding

I want to...	Open
change how a page looks / add a button	`client/AppClient/Pages/<Page>.cs` (double-click for the designer)
add a page to the menu	the `menu` list in `client/AppClient/Forms/MainForm.cs`
write a function	`client/AppClient/Helpers/InventoryMath.cs`
call the server from C#	`client/AppClient/Services/ApiClient.cs`
add an API endpoint	`server/src/routes` → `controllers` → `models`
add or change a table	a new numbered file in `database/migrations/`

Get the code

```
git clone https://github.com/Shaloh69/AGP-Inventory-System.git
cd AGP-Inventory-System
```

SECTION 5

Project Plan and Timeline

A suggested 8-week plan for 3–5 people. The template already covers the hardest set-up work, so the weeks go into features.

Phase	Wk	Goal	Main tasks	Done when…
0 · Setup	1	Everyone runs it	Install tools, clone, run-dev.bat check, read Sections 1–10	Everyone has the app running locally
1 · Plan	1	Know what to build	Feature list, screen sketches, ERD, endpoint list	Team agrees on a one-page plan
2 · Database	2	Aiven ready	Aiven service, .env + ca.pem, migrate + seed	Products page shows Aiven data
3 · Deploy	2	API online	Render web service, env vars, secret file	/api/health works on onrender.com
4 · UI choice	2	One look	Compare library pages, pick one, restyle the starter pages	Team agrees on one library
5 · Features	3–6	The inventory	Suppliers, categories, stock in/out, reports – one feature per branch	Each feature works end to end
6 · Login	6–7	Secure	Users table, bcrypt + JWT (Section 22)	Wrong password is rejected
7 · Finish	8	Ready to submit	Testing, fixes, docs, demo rehearsal	Final checklist (Section 24) complete

Suggested roles

Role	Responsible for	Folders
Project lead	Plan, reviewing pull requests, Render/Aiven accounts	everything
Database + backend	Migrations, models, controllers, routes, deployment	database/, server/
Frontend (1–2 people)	Pages, UI library, ApiClient methods	client/
QA / docs	Testing every feature, docs, demo	docs/

Feature ideas for an inventory system

- **Suppliers** – who you buy from (built step by step in Section 17).
- **Categories** – group products (Tools, Paint, Hardware): a `categories` table + `category_id` on products.
- **Stock movements** – every stock in / stock out with date and reason, updating `products.quantity`.
- **Low-stock report** – reuse IsLowStock() and show only the products that need re-ordering.
- **Users and login** – Section 22.

SECTION 6

Set Up the Database on Aiven

One person does this once, then shares the connection details privately with the team.

Step 1 – Create the service

1. Log in to **console.aiven.io** → **Create service** → **PostgreSQL**.
2. Choose the **Free** plan and a region close to the Render region you will use.
3. Name it (e.g. *agp-db*) and wait until the status is **Running**.

Step 2 – Connect the server to it

From the Aiven Overview page	Put it in
Service URI (postgres://avnadmin:...@...aivencloud.com:PORT/defaultdb?sslmode=require)	`server/.env` as `DATABASE_URL` – **delete** `?sslmode=require` from the end
CA certificate (Download)	`server/certs/ca.pem`

server/.env (copy of .env.example, filled in)

```
PORT=3000

# From Aiven Console -> your PostgreSQL service -> Overview -> Service URI
# IMPORTANT: remove "?sslmode=require" from the end; SSL is configured in src/config/db.js
DATABASE_URL=postgres://avnadmin:YOUR_PASSWORD@YOUR_HOST.aivencloud.com:12345/defaultdb

# Path to the Aiven CA certificate (download "CA certificate" from the Aiven Overview page)
# Local:  ./certs/ca.pem
# Render: /etc/secrets/ca.pem   (upload it as a Secret File named ca.pem)
DB_CA_PATH=./certs/ca.pem
```

The connection itself is in `server/src/config/db.js`. It turns on SSL and checks Aiven's certificate:

server/src/config/db.js

```
function sslConfig() {
  if (process.env.DB_SSL === 'false') return false;

  const caPath = process.env.DB_CA_PATH;
  if (!caPath || !fs.existsSync(caPath)) {
    throw new Error(
      `CA certificate not found at "${caPath}". Download it from the Aiven console ` +
      'and save it as server/certs/ca.pem (or set DB_SSL=false for a local database).'
    );
  }
  return { rejectUnauthorized: true, ca: fs.readFileSync(path.resolve(caPath)).toString() };
}

if (!process.env.DATABASE_URL) {
  throw new Error('DATABASE_URL is missing. Copy server/.env.example to server/.env and fill it
      in.');
}

// A Pool keeps a few connections open and reuses them (much faster than reconnecting).
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: sslConfig(),
});

module.exports = pool;
```

WATCH OUT The Service URI contains the database password. Share it only in a private chat – never in GitHub, screenshots or group chats.

Step 3 – Create the tables and sample data

Terminal

```
run-dev.bat setup          (from the repository root)

  ...or by hand:
cd server
npm install
npm run db:migrate         ran   001_create_products_table.sql
npm run db:seed            seeded 001_sample_products.sql
```

`db:migrate` runs every file in `database/migrations` that has not run yet and remembers it in a `schema_migrations` table, so running it again is safe. Section 16 explains migrations in detail.

TIP Optional: connect DBeaver (New connection → PostgreSQL, host/port/user/password from Aiven, SSL mode “require”) and run SELECT * FROM products; to see the five sample products.

SECTION 7

Run Everything on Your PC

One command starts the server and the app. Use it every time you want to test a change.

Command (repository root)	What it does
`run-dev.bat check`	Checks git, node, npm, dotnet, .env, ca.pem and installed packages
`run-dev.bat setup`	First time: npm install, migrations, sample data, C# packages
`run-dev.bat`	Starts the server in its own window, waits until it answers, then opens the app
`run-dev.bat server`	Only the server (http://localhost:3000). It restarts by itself when you save a .js file
`run-dev.bat client`	Only the app, pointed at the local server
`run-dev.bat client Products`	The app, opened straight on one page (Home, Products, MahApps ...)

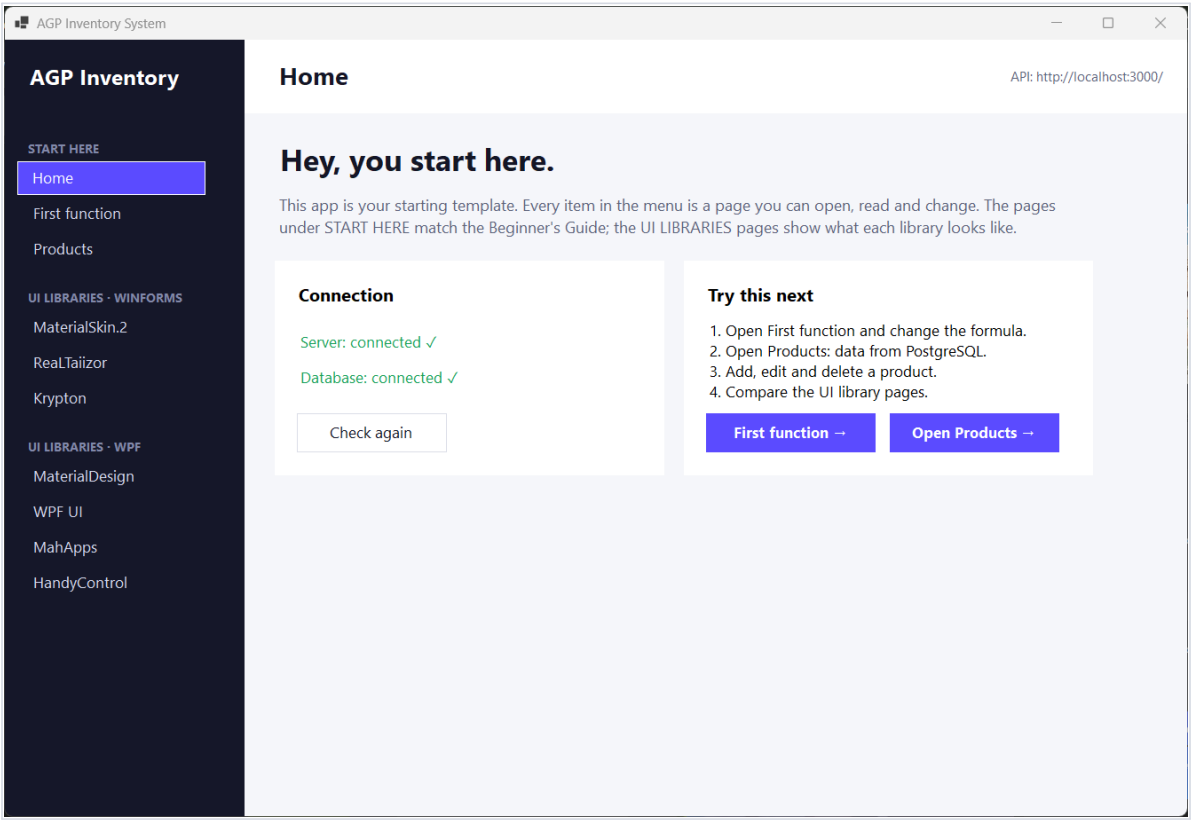


Figure 2 – The Home page: both lines green means the app → server → database chain works

How does the app know to use your PC's server? `run-dev.bat` sets `API_BASE_URL`, and `AppConfig` reads it; without it, the app uses the Render address:

client/AppClient/Helpers/AppConfig.cs

```
public static class AppConfig
{
    // run-dev.bat sets API_BASE_URL=http://localhost:3000/ for local testing.
    // Otherwise the app uses the live Render server. The URL MUST end with "/".
    public static readonly string ApiBaseUrl =
        Environment.GetEnvironmentVariable("API_BASE_URL")
        ?? "https://agp-api.onrender.com/";
}
```

Testing without Aiven (optional)

If you install PostgreSQL on your own PC, point `DATABASE_URL` at it and add `DB_SSL=false` to `.env`. No certificate is needed. This is handy when the internet is slow, but the team's real data stays on Aiven.

server/.env for a local PostgreSQL

```
DATABASE_URL=postgres://postgres:yourpassword@localhost:5432/agp
DB_SSL=false
```

Running from Visual Studio

Open `client/AppClient.slnx` and press **F5**. To use the local server from Visual Studio, set the environment variable in **Project → Properties → Debug → Open debug launch profiles UI**: `API_BASE_URL` = `http://localhost:3000/`. To start on a page, add command line arguments `--page Products`.

SECTION 8

How the Server Works

Every request goes through the same four files. Learn them once and every new feature is copy, paste, change.

The request flow

```
request  –  app.js
              └─ routes/product.routes.js          which function handles this URL?
                    └─ controllers/product.controller.js   check input, pick status code
                          └─ models/product.model.js         run the SQL
                                └─ PostgreSQL
response  ── JSON  ── controller  (res.json / res.status(...).json)
```

File	Its one job
src/index.js	Loads .env and starts listening on a port
src/app.js	Adds middleware (JSON, CORS), connects routes, adds error handlers last
routes/*.routes.js	Maps a URL + method to a controller function
controllers/*.controller.js	Reads the request, validates it, calls the model, chooses the status code
models/*.model.js	The only place with SQL for that table
middleware/errorHandler.js	Turns any thrown error into a JSON message

server/src/app.js

```
const app = express();
app.use(cors());
app.use(express.json()); // lets us read JSON bodies as req.body

// Is the server running?
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));

// Can the server reach the database?
app.get('/api/health/db', async (req, res) => {
  const result = await pool.query('SELECT NOW() AS time');
  res.json({ status: 'ok', databaseTime: result.rows[0].time });
});

app.use('/api/products', productRoutes);

app.use(notFound);
app.use(errorHandler); // always last

module.exports = app;
```

server/src/routes/product.routes.js

```
const router = express.Router();

router.get('/', controller.getAll);        // GET    /api/products  (or /api/products?search=ham)
router.get('/:id', controller.getOne);     // GET    /api/products/5
router.post('/', controller.create);       // POST   /api/products
router.put('/:id', controller.update);     // PUT    /api/products/5
router.delete('/:id', controller.remove);  // DELETE /api/products/5

module.exports = router;
```

server/src/middleware/errorHandler.js

```
// Express calls this whenever a route throws an error (Express 5 also catches async errors).
function errorHandler(err, req, res, next) {
  // PostgreSQL error codes: https://www.postgresql.org/docs/current/errcodes-appendix.html
  if (err.code === '23505') return res.status(409).json({ message: 'That name already exists' });
  if (err.code === '22P02') return res.status(400).json({ message: 'Invalid id or value' });

  console.error(err); // shows in the terminal and in the Render logs
  res.status(err.status || 500).json({ message: 'Something went wrong on the server' });
}

module.exports = errorHandler;
```

Express 5 sends errors thrown inside `async` functions straight to this handler, so controllers need no try/catch. Because `products.name` is `UNIQUE`, saving a duplicate name throws PostgreSQL error `23505`, which becomes a friendly 409.

All endpoints

Method	URL	Body	Success
GET	`/api/health`	–	`{ "status": "ok" }`
GET	`/api/health/db`	–	200 + database time
GET	`/api/products?search=ham`	–	200 + array
GET	`/api/products/1`	–	200 + product, or 404
POST	`/api/products`	`{ "name", "quantity", "price" }`	201 + new product
PUT	`/api/products/1`	`{ "name", "quantity", "price" }`	200 + updated product
DELETE	`/api/products/1`	–	204 (no body)

TIP Open http://localhost:3000/api/products in a browser while the server runs – you will see the JSON the app receives. Use Postman or Bruno for POST, PUT and DELETE.

SECTION 9

Deploy the API on Render

Render takes the server from GitHub, runs npm install and npm start, and gives you a public HTTPS address.

1. Push the latest code to GitHub (branch main). Check that .env and ca.pem are **not** in the repository.
2. Render dashboard → **New +** → **Web Service** → connect the repository.
3. Fill in the settings below, choose the **Free** instance, add the environment variables and secret file.
4. Click **Create Web Service** and watch the logs until you see "AGP API running".

Setting	Value
Name	agp-api (becomes agp-api.onrender.com)
Root Directory	`server` (important: the server is in a sub-folder)
Build Command / Start Command	`npm install` / `npm start`
Health Check Path	`/api/health`
Environment variables	`DATABASE_URL` (same as .env) · `DB_CA_PATH` = `/etc/secrets/ca.pem`
Secret File	name `ca.pem`, paste the whole Aiven certificate

- Do **not** set `PORT`; Render sets it.
- Every push to main redeploys automatically – test locally first.
- The **Logs** tab shows console.log output and errors.
- Free services sleep after ~15 minutes idle; the first request can take up to a minute. The app's 90-second timeout covers this. Warm it up before a demo.
- If you chose another name, change the Render URL in `Helpers/AppConfig.cs`.

CHECKPOINT https://agp-api.onrender.com/api/health/db answers {"status":"ok",...} and the app, started from Visual Studio without API_BASE_URL, shows Aiven's products.

SECTION 10

How the C# App Works

A main window with a sidebar. Clicking a menu item creates a page and shows it on the right.

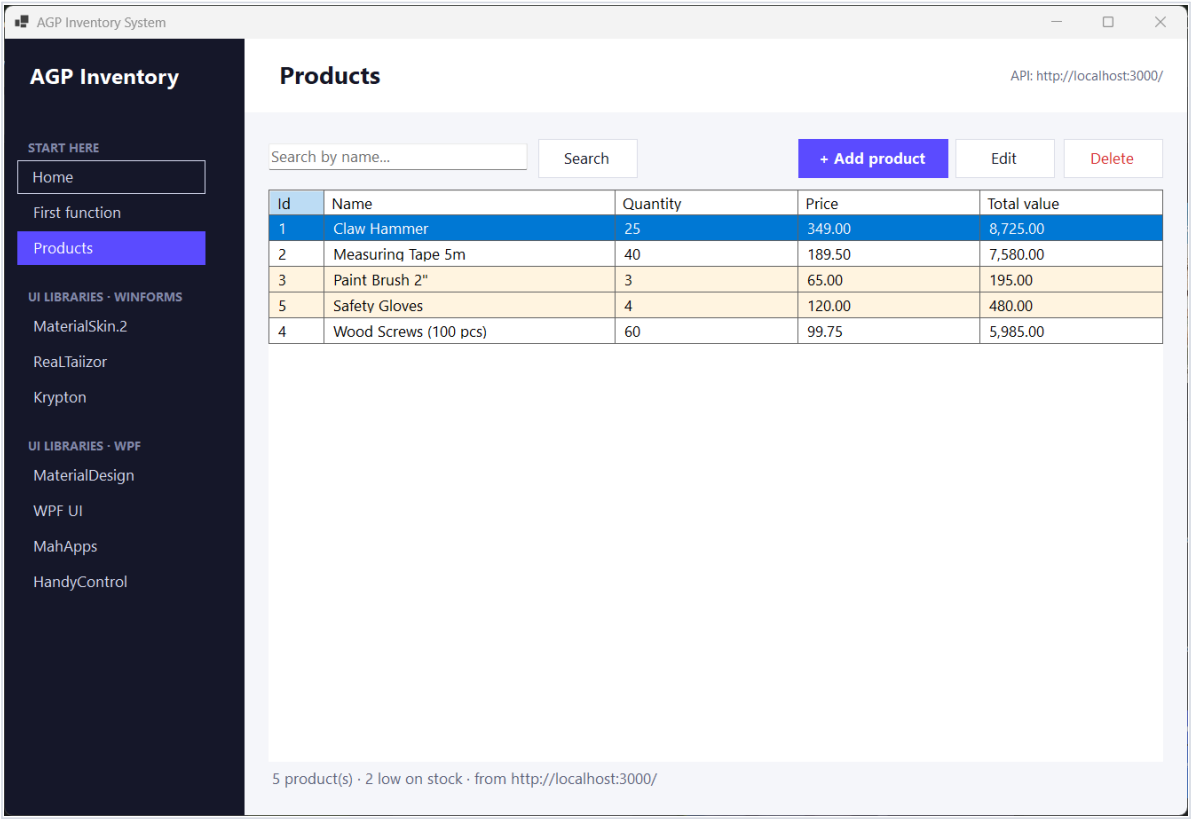


Figure 3 – The main window (Forms/MainForm.cs) showing the Products page

Part	File	Job
Main window	`Forms/MainForm.cs`	Builds the sidebar from the menu list and swaps pages
Pages	`Pages/*.cs`	One UserControl per screen
Navigator	`Helpers/Navigator.cs`	Lets any page open any other page
ApiClient	`Services/ApiClient.cs`	Every HTTP call to the server
Models	`Models/Product.cs`	C# classes shaped like the JSON
Helpers	`Helpers/InventoryMath.cs`, `Theme.cs`	Functions and shared colours/fonts

The sidebar is built from one list. Each line is a title and a small function that creates the page (`() => new HomePage()`). Section headings use `MenuItem.Section`:

client/AppClient/Forms/MainForm.cs

```
private readonly List<MenuItem> menu =
[
    MenuItem.Section("START HERE"),
    new("Home", () => new HomePage()),
    new("First function", () => new FirstFunctionPage()),
    new("Products", () => new ProductsPage()),

    MenuItem.Section("UI LIBRARIES · WINFORMS"),
    new("MaterialSkin.2", () => new MaterialSkinPage()),
    new("ReaLTaiizor", () => new ReaLTaiizorPage()),
    new("Krypton", () => new KryptonPage()),

    MenuItem.Section("UI LIBRARIES · WPF"),
    new("MaterialDesign", () => new WpfHostPage(new MaterialDesignPage())),
    new("WPF UI", () => new WpfHostPage(new WpfUiPage())),
    new("MahApps", () => new WpfHostPage(new MahAppsPage())),
    new("HandyControl", () => new WpfHostPage(new HandyControlPage())),
];
```

Showing a page removes the old one, docks the new one to fill the area, and sets the title:

client/AppClient/Forms/MainForm.cs

```
public void ShowPage(UserControl page, string title)
{
    foreach (Control old in panelContent.Controls.Cast<Control>().ToList())
    {
        panelContent.Controls.Remove(old);
        old.Dispose();
    }
```

TIP The WPF library pages live in a second project, `AppClient.WpfPages`, and are shown inside the WinForms window by `WpfHostPage` (an ElementHost). You only need this if you choose a WPF library.

SECTION 11

How To: Write Your First Function

A function takes values in, does one job, and gives a result back. Start with small functions that do not touch the screen or the internet.

client/AppClient/Helpers/InventoryMath.cs

```
public static class InventoryMath
{
    public const int LowStockLimit = 5;

    // Total value of the stock on hand: price x quantity.
    public static decimal CalculateTotal(decimal price, int quantity)
    {
        return price * quantity;
    }

    // true when the item should be re-ordered soon.
    public static bool IsLowStock(int quantity)
    {
        return quantity <= LowStockLimit;
    }
}
```

Part	Example	Meaning
access	`public`	Other classes may call it
static	`static`	Call it on the class: InventoryMath.CalculateTotal(...) – no object needed
return type	`decimal` / `bool`	What comes back (decimal for money, bool for yes/no)
name	`CalculateTotal`	PascalCase, a verb that says what it does
parameters	`(decimal price, int quantity)`	The values that go in
body	`return price * quantity;`	The work, ending with return

Calling it from a button

The First function page reads the inputs, calls both functions, and shows the results:

client/AppClient/Pages/FirstFunctionPage.cs

```
private void btnCalculate_Click(object? sender, EventArgs e)
{
    // 1. Read the inputs
    decimal price = numPrice.Value;
    int quantity = (int)numQuantity.Value;

    // 2. Call the functions
    decimal total = InventoryMath.CalculateTotal(price, quantity);
    bool lowStock = InventoryMath.IsLowStock(quantity);

    // 3. Show the results
    lblResult.Text = $"Total: {total:N2}";
    lblStock.Text = lowStock ? "  Low stock - time to re-order" : "  Stock level is fine";
    lblStock.ForeColor = lowStock ? Theme.Danger : Theme.Success;
}
```

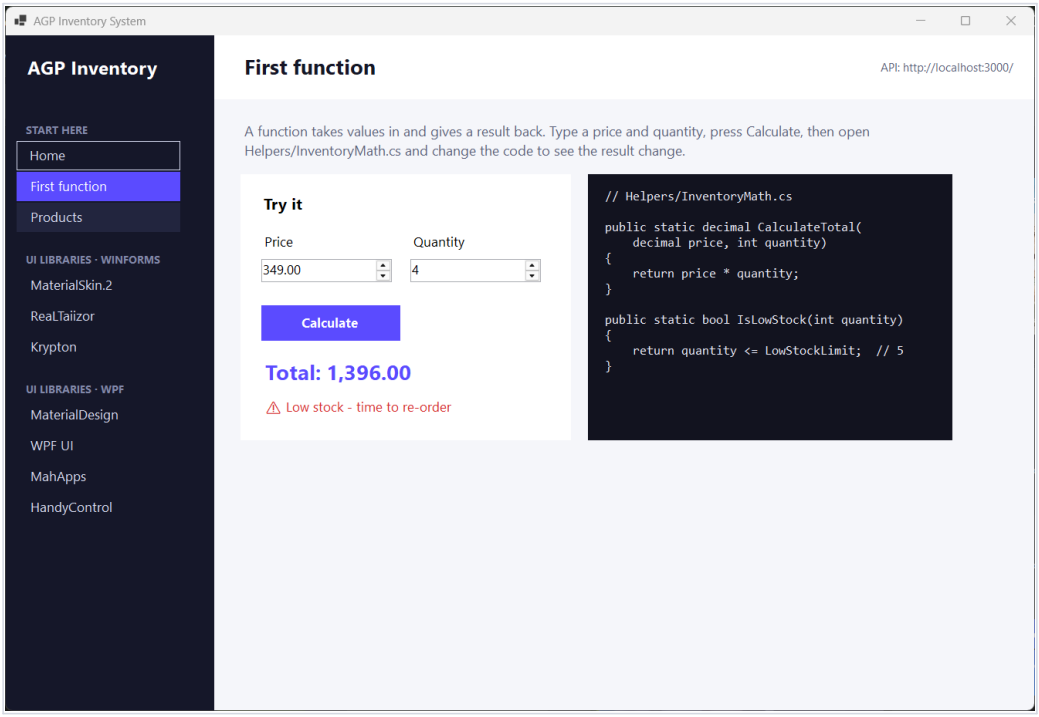


Figure 4 – The First function page

Exercise – write your own

Add this under `IsLowStock` in InventoryMath.cs, then show "Out of stock" on the First function page when it returns true:

client/AppClient/Helpers/InventoryMath.cs

```
    // true when there is nothing left on the shelf.
    public static bool IsOutOfStock(int quantity)
    {
        return quantity == 0;
    }
```

Functions are reused everywhere: the Products page uses `IsLowStock` to colour rows, and `Product.TotalValue` uses `CalculateTotal`.

A first function in JavaScript

The server has the same idea. This pure function checks a product and returns the error messages:

server/src/utils/validateProduct.js

```
function validateProduct(body) {
  const errors = [];
  const name = typeof body.name === 'string' ? body.name.trim() : '';
  const quantity = Number(body.quantity);
  const price = Number(body.price);

  if (name === '') errors.push('name is required');
  if (name.length > 120) errors.push('name must be 120 characters or less');
  if (!Number.isInteger(quantity) || quantity < 0) errors.push('quantity must be a whole number, 0
      or more');
  if (Number.isNaN(price) || price < 0) errors.push('price must be a number, 0 or more');

  return { errors, product: { name, quantity, price } };
}

module.exports = validateProduct;
```

SECTION 12

How To: Create a Page

A page is a UserControl: a piece of screen you design once and show inside the main window.

1. Visual Studio → **Solution Explorer** → right-click the **Pages** folder → **Add** → **User Control (Windows Forms)**.
2. Name it after what it shows, ending in Page: `SuppliersPage.cs`.
3. Drag controls from the **Toolbox** (View → Toolbox) and set their properties in the **Properties** window (F4).
4. Give every control you use in code a clear **(Name)**: `txtName`, `btnAdd`, `dgvSuppliers`.
5. Double-click a button to create its Click method, then write the code.

For the Suppliers page in Section 17 you drag these controls:

Toolbox control	(Name)	Properties
TextBox	`txtName`	PlaceholderText = Supplier name
TextBox	`txtPhone`	PlaceholderText = Phone
Button	`btnAdd`	Text = Add supplier; double-click it to create btnAdd_Click
DataGridView	`dgvSuppliers`	Dock = Bottom (or Fill), ReadOnly = True
Label	`lblStatus`	Text = Loading…

Visual Studio writes the layout into `SuppliersPage.Designer.cs` for you – never edit that file by hand. You write your code in `SuppliersPage.cs`:

client/AppClient/Pages/SuppliersPage.cs

```
using AppClient.Helpers;
using AppClient.Models;
using AppClient.Services;

namespace AppClient.Pages;

public partial class SuppliersPage : UserControl
{
    public SuppliersPage()
    {
        InitializeComponent();
        Theme.StylePrimary(btnAdd);
    }

    // Runs once when the page is shown: load the list.
    protected override async void OnLoad(EventArgs e)
    {
        base.OnLoad(e);
        await LoadSuppliersAsync();
    }

    private async Task LoadSuppliersAsync()
    {
        try
        {
            List<Supplier> suppliers = await ApiClient.GetSuppliersAsync();
            dgvSuppliers.DataSource = suppliers;
            lblStatus.Text = $"{suppliers.Count} supplier(s)";
        }
        catch (Exception ex)
        {
            lblStatus.Text = "Could not load suppliers: " + ex.Message;
        }
    }

    private async void btnAdd_Click(object? sender, EventArgs e)
    {
        try
        {
            var supplier = new Supplier { Name = txtName.Text.Trim(), Phone = txtPhone.Text.Trim()
                };
            await ApiClient.CreateSupplierAsync(supplier);
            txtName.Clear();
            txtPhone.Clear();
            await LoadSuppliersAsync();
        }
        catch (Exception ex)
        {
            lblStatus.Text = "Could not save: " + ex.Message;
        }
    }
}
```

Tip	Why
Load data in `OnLoad`, not in the constructor	The page is on screen, so you can show “Loading…” and errors
Use `async` / `await` for every server call	The window stays responsive while waiting

Tip	Why
Wrap server calls in `try / catch`	No crash when the server is down; show a message instead
Set `Dock` or `Anchor` on grids and panels	The page still looks right when the window is resized
Style buttons with `Theme.StylePrimary(btnAdd)`	Every page looks the same

SECTION 13

How To: Open a Page and Pass Data

There are three ways to show a page: from the menu, from a button, and with data passed in.

1. From the sidebar menu

Add one line to the `menu` list in `Forms/MainForm.cs`. The position in the list is the position in the sidebar:

client/AppClient/Forms/MainForm.cs

```
new("Products", () => new ProductsPage()),
new("Suppliers", () => new SuppliersPage()),      // ← new
```

2. From a button on another page

Create the page and give it to `Navigator.Go` with the title to show:

client/AppClient/Pages/HomePage.cs

```
// How to open (call) another page: create it and pass it to Navigator.Go
private void btnGoFunction_Click(object? sender, EventArgs e) => Navigator.Go(new
    FirstFunctionPage(), "First function");

private void btnGoProducts_Click(object? sender, EventArgs e) => Navigator.Go(new ProductsPage(),
    "Products");
```

client/AppClient/Helpers/Navigator.cs

```
public static class Navigator
{
    private static MainForm? shell;

    // Called once by MainForm when the app starts.
    public static void Attach(MainForm mainForm) => shell = mainForm;

    public static void Go(UserControl page, string title)
    {
        if (shell == null) throw new InvalidOperationException("MainForm has not started yet.");
        shell.ShowPage(page, title);
    }
}
```

3. Passing data to a page

Give the page a constructor parameter. `ProductFormPage` takes an optional product: none means “Add”, a product means “Edit” and fills the form:

client/AppClient/Pages/ProductFormPage.cs

```
private readonly Product? editing; // null = adding a new product

public ProductFormPage(Product? product = null)
{
    InitializeComponent();
    Theme.StylePrimary(btnSave);
    Theme.StyleSecondary(btnCancel);

    editing = product;
    if (editing != null)
    {
        // Data passed in from ProductsPage fills the form.
        lblHeading.Text = $"Edit product #{editing.Id}";
        txtName.Text = editing.Name;
        numQuantity.Value = editing.Quantity;
        numPrice.Value = editing.Price;
    }
    UpdateTotal();
```

The Products page passes the selected row's product when you click Edit (or double-click a row):

client/AppClient/Pages/ProductsPage.cs

```
private void btnAdd_Click(object? sender, EventArgs e) =>
    Navigator.Go(new ProductFormPage(), "Add product");

private void btnEdit_Click(object? sender, EventArgs e)
{
    if (SelectedProduct is not Product product)
    {
        MessageBox.Show("Select a product first.", "Edit", MessageBoxButtons.OK,
            MessageBoxIcon.Information);
        return;
    }
    Navigator.Go(new ProductFormPage(product), "Edit product");
```

To go back, open the previous page again – for example after saving: `Navigator.Go(new ProductsPage(), "Products");`

CHECKPOINT Run `run-dev.bat client Products`, select a product, click Edit: the heading shows “Edit product #id” and the form is filled. That is data passed between pages.

SECTION 14

How To: Call the Database (Read)

The app never talks to PostgreSQL. It asks the API, the API runs SQL, and the rows come back as JSON.

The read path, top to bottom

```
ProductsPage.LoadProductsAsync()
   └─ ApiClient.GetProductsAsync(search)          C#    GET /api/products?search=...
         └─ product.routes.js  router.get('/')      JS
               └─ controller.getAll(req, res)
                     └─ Product.findAll(search)   SQL   SELECT * FROM products ...
```

Server side: the SQL

server/src/models/product.model.js

```
async function findAll(search) {
  if (search) {
    const result = await pool.query(
      `SELECT * FROM products WHERE name ILIKE '%' || $1 || '%' ORDER BY name`,
      [search]
    );
    return result.rows;
  }
  const result = await pool.query('SELECT * FROM products ORDER BY name');
  return result.rows;
}
```

server/src/controllers/product.controller.js

```
async function getAll(req, res) {
  const products = await Product.findAll(req.query.search);
  res.json(products);
}
```

Client side: the model

The C# class has one property per JSON field. `[JsonPropertyName]` connects the database's snake_case (`created_at`) to C#'s PascalCase (`CreatedAt`). `[Browsable(false)]` hides a column in the grid:

client/AppClient/Models/Product.cs

```
public class Product
{
    [JsonPropertyName("id")] public int Id { get; set; }
    [JsonPropertyName("name")] public string Name { get; set; } = "";
    [JsonPropertyName("quantity")] public int Quantity { get; set; }
    [JsonPropertyName("price")] public decimal Price { get; set; }

    [JsonPropertyName("created_at"), Browsable(false)] public DateTime CreatedAt { get; set; }
    [JsonPropertyName("updated_at"), Browsable(false)] public DateTime UpdatedAt { get; set; }

    // Calculated in C#, not stored in the database. [JsonIgnore] = never sent to the API.
    [JsonIgnore, DisplayName("Total value")]
    public decimal TotalValue => InventoryMath.CalculateTotal(Price, Quantity);
}
```

Client side: the call

client/AppClient/Services/ApiClient.cs – one HttpClient for the whole app

```
private static readonly HttpClient http = new()
{
    BaseAddress = new Uri(AppConfig.ApiBaseUrl),
    Timeout = TimeSpan.FromSeconds(90) // Render's free plan can take a while to wake up
};
```

client/AppClient/Services/ApiClient.cs

```
public static async Task<List<Product>> GetProductsAsync(string? search = null)
{
    string url = string.IsNullOrWhiteSpace(search)
        ? "api/products"
        : $"api/products?search={Uri.EscapeDataString(search.Trim())}";
    var response = await http.GetAsync(url);
    await EnsureSuccessAsync(response);
    return await response.Content.ReadFromJsonAsync<List<Product>>() ?? new();
}
```

Client side: showing the rows

client/AppClient/Pages/ProductsPage.cs

```
private async Task LoadProductsAsync()
{
    lblStatus.Text = "Loading…";
    try
    {
        List<Product> products = await ApiClient.GetProductsAsync(txtSearch.Text);
        dgvProducts.DataSource = products;
        FormatColumns();
        int low = products.Count(p => InventoryMath.IsLowStock(p.Quantity));
        lblStatus.Text = $"{products.Count} product(s) · {low} low on stock · from
            {ApiClient.BaseUrl}";
    }
```

Why async / await?

A network call can take seconds. `await` lets the window keep drawing while it waits; without it Windows shows “Not responding”. Rule: event handlers may be `async void`; every other async method returns `Task` or `Task<T>` and its name ends with `Async`. Never use `.Result` or `.Wait()` – they freeze the app.

SECTION 15

How To: Store Data (Create, Update, Delete)

Storing is the read path in reverse: the page fills an object, ApiClient sends it, the model runs INSERT / UPDATE / DELETE.

The Save button

client/AppClient/Pages/ProductFormPage.cs

```
private async void btnSave_Click(object? sender, EventArgs e)
{
    lblError.Text = "";

    // 1. Check the input before calling the server
    if (string.IsNullOrWhiteSpace(txtName.Text))
    {
        lblError.Text = "Please type a product name.";
        txtName.Focus();
        return;
    }

    // 2. Copy the form into a Product object
    var product = new Product
    {
        Id = editing?.Id ?? 0,
        Name = txtName.Text.Trim(),
        Quantity = (int)numQuantity.Value,
        Price = numPrice.Value,
    };

    // 3. Save it
    btnSave.Enabled = false;
    try
    {
        if (editing == null)
            await ApiClient.CreateProductAsync(product);   // POST /api/products
        else
            await ApiClient.UpdateProductAsync(product);   // PUT  /api/products/{id}

        // 4. Go back to the list
        Navigator.Go(new ProductsPage(), "Products");
    }
    catch (ApiException ex)
    {
        lblError.Text = ex.Message; // e.g. "That name already exists"
    }
    catch (Exception ex)
    {
        lblError.Text = "Could not reach the server: " + ex.Message;
    }
    finally
    {
        if (!IsDisposed) btnSave.Enabled = true;
    }
}
```

ApiClient: POST, PUT, DELETE and error messages

client/AppClient/Services/ApiClient.cs

```
public static async Task<Product> CreateProductAsync(Product product)
{
    var response = await http.PostAsJsonAsync("api/products", product);
    await EnsureSuccessAsync(response);
    return (await response.Content.ReadFromJsonAsync<Product>())!;
}

public static async Task<Product> UpdateProductAsync(Product product)
{
    var response = await http.PutAsJsonAsync($"api/products/{product.Id}", product);
    await EnsureSuccessAsync(response);
    return (await response.Content.ReadFromJsonAsync<Product>())!;
}

public static async Task DeleteProductAsync(int id)
{
    var response = await http.DeleteAsync($"api/products/{id}");
    await EnsureSuccessAsync(response);
}

// Turns an error response like {"message":"name is required"} into an ApiException.
private static async Task EnsureSuccessAsync(HttpResponseMessage response)
{
    if (response.IsSuccessStatusCode) return;

    string message = response.ReasonPhrase ?? "Request failed";
    try
    {
        var error = await response.Content.ReadFromJsonAsync<ApiError>();
        if (!string.IsNullOrWhiteSpace(error?.Message)) message = error.Message;
    }
    catch (Exception)
    {
        // the body was not JSON - keep the default message
    }
    throw new ApiException((int)response.StatusCode, message);
}
```

When the server answers with an error such as `{"message":"That name already exists"}`, `EnsureSuccessAsync` throws an `ApiException` with that message, and the page shows it in `lblError`.

Server: validate, then save

server/src/controllers/product.controller.js

```
async function create(req, res) {
  const { errors, product } = validateProduct(req.body);
  if (errors.length > 0) return res.status(400).json({ message: errors.join(', ') });

  const saved = await Product.create(product);
  res.status(201).json(saved);
}

async function update(req, res) {
  const { errors, product } = validateProduct(req.body);
  if (errors.length > 0) return res.status(400).json({ message: errors.join(', ') });

  const saved = await Product.update(req.params.id, product);
  if (!saved) return res.status(404).json({ message: 'Product not found' });
  res.json(saved);
}
```

server/src/models/product.model.js

```
async function create({ name, quantity, price }) {
  const result = await pool.query(
    `INSERT INTO products (name, quantity, price)
     VALUES ($1, $2, $3)
     RETURNING *`,
    [name, quantity, price]
  );
  return result.rows[0];
}

async function update(id, { name, quantity, price }) {
  const result = await pool.query(
    `UPDATE products
        SET name = $1, quantity = $2, price = $3, updated_at = NOW()
      WHERE id = $4
      RETURNING *`,
    [name, quantity, price, id]
  );
  return result.rows[0];
}

async function remove(id) {
  const result = await pool.query('DELETE FROM products WHERE id = $1', [id]);
  return result.rowCount > 0;
}
```

Delete with a confirmation

client/AppClient/Pages/ProductsPage.cs

```
private async void btnDelete_Click(object? sender, EventArgs e)
{
    if (SelectedProduct is not Product product) return;

    var answer = MessageBox.Show($"Delete \"{product.Name}\"?", "Delete product",
        MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
    if (answer != DialogResult.Yes) return;

    try
    {
        await ApiClient.DeleteProductAsync(product.Id);
        await LoadProductsAsync();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Could not delete: " + ex.Message, "Error", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

Try this in the app	What you should see
Add a product with an empty name	“Please type a product name.” (checked in C#, no server call)
Add a product whose name already exists	“That name already exists” (409 from the server)
Edit a product's quantity to 3	Back on Products, the row is orange (IsLowStock)
Delete it and answer Yes	The row disappears; it is gone from PostgreSQL too

SECTION 16

How To: Add or Change a Table (Migrations)

Tables are created and changed only by numbered SQL files, so every computer and Aiven end up with the same structure.

database/migrations/001_create_products_table.sql

```
-- 001: the products table (the first feature of the inventory system)
CREATE TABLE products (
    id          SERIAL        PRIMARY KEY,
    name        VARCHAR(120)  NOT NULL UNIQUE,
    quantity    INTEGER       NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    price       NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (price >= 0),
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Rule	Example
One change per file, numbered in order	`002_create_suppliers_table.sql`, `003_add_category_to_products.sql`
Never edit a file that has already run – add a new one	`ALTER TABLE products ADD COLUMN category_id INTEGER;`
Run it on every database (yours, then Aiven)	`npm run db:migrate`
Sample data goes in seeds, written to be safe to repeat	`... ON CONFLICT (name) DO NOTHING;`

database/seeds/001_sample_products.sql

```
-- Sample products for testing. ON CONFLICT DO NOTHING makes it safe to run again.
INSERT INTO products (name, quantity, price) VALUES
    ('Claw Hammer',          25, 349.00),
    ('Measuring Tape 5m',    40, 189.50),
    ('Paint Brush 2"',        3,  65.00),
    ('Wood Screws (100 pcs)', 60,  99.75),
    ('Safety Gloves',         4, 120.00)
ON CONFLICT (name) DO NOTHING;
```

Common column types

PostgreSQL	Use for	C# type
`SERIAL` / `INTEGER`	ids, counts, quantities	`int`
`VARCHAR(n)` / `TEXT`	names, notes	`string`
`NUMERIC(10,2)`	money (never FLOAT)	`decimal`
`BOOLEAN`	yes / no	`bool`
`DATE` / `TIMESTAMPTZ`	dates / exact moments	`DateTime`
`INTEGER REFERENCES suppliers(id)`	a link to another table	`int`

SECTION 17

Tutorial: Build a Complete Feature (Suppliers)

Ten steps from an empty idea to a working page. Every snippet below was applied to a copy of this repository, built and run before printing.

Step 1 – The table

database/migrations/002_create_suppliers_table.sql

```
-- 002: suppliers we buy products from
CREATE TABLE suppliers (
    id          SERIAL        PRIMARY KEY,
    name        VARCHAR(120)  NOT NULL UNIQUE,
    phone       VARCHAR(30),
    created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);
```

Terminal

```
cd server
npm run db:migrate          ran   002_create_suppliers_table.sql
```

Step 2 – The model (SQL)

server/src/models/supplier.model.js

```
const pool = require('../config/db');

async function findAll() {
  const result = await pool.query('SELECT * FROM suppliers ORDER BY name');
  return result.rows;
}

async function create({ name, phone }) {
  const result = await pool.query(
    'INSERT INTO suppliers (name, phone) VALUES ($1, $2) RETURNING *',
    [name, phone]
  );
  return result.rows[0];
}

module.exports = { findAll, create };
```

Step 3 – The controller

server/src/controllers/supplier.controller.js

```
const Supplier = require('../models/supplier.model');

async function getAll(req, res) {
  res.json(await Supplier.findAll());
}

async function create(req, res) {
  const name = typeof req.body.name === 'string' ? req.body.name.trim() : '';
  if (name === '') return res.status(400).json({ message: 'name is required' });

  const saved = await Supplier.create({ name, phone: req.body.phone || null });
  res.status(201).json(saved);
}

module.exports = { getAll, create };
```

Step 4 – The routes

server/src/routes/supplier.routes.js

```
const express = require('express');
const controller = require('../controllers/supplier.controller');

const router = express.Router();

router.get('/', controller.getAll);    // GET  /api/suppliers
router.post('/', controller.create);   // POST /api/suppliers

module.exports = router;
```

Step 5 – Connect the routes in app.js

server/src/app.js

```
app.use('/api/products', productRoutes);
app.use('/api/suppliers', require('./routes/supplier.routes'));      // ← new
```

Step 6 – Test the API

Request	Expected
`GET http://localhost:3000/api/suppliers`	`200` and `[]` (empty list)
`POST /api/suppliers { "name": "ACME Tools", "phone": "0917 000 0000" }`	`201` and the new supplier with its id
`POST /api/suppliers { "name": "" }`	`400` name is required
The same name again	`409` That name already exists (the shared error handler)

Step 7 – The C# model

client/AppClient/Models/Supplier.cs

```
using System.ComponentModel;
using System.Text.Json.Serialization;

namespace AppClient.Models;

public class Supplier
{
    [JsonPropertyName("id")] public int Id { get; set; }
    [JsonPropertyName("name")] public string Name { get; set; } = "";
    [JsonPropertyName("phone")] public string? Phone { get; set; }
    [JsonPropertyName("created_at"), Browsable(false)] public DateTime CreatedAt { get; set; }
}
```

Step 8 – ApiClient methods

Add these inside the `ApiClient` class, above `EnsureSuccessAsync`:

client/AppClient/Services/ApiClient.cs

```
    // ---------- suppliers ----------
    public static async Task<List<Supplier>> GetSuppliersAsync()
    {
        var response = await http.GetAsync("api/suppliers");
        await EnsureSuccessAsync(response);
        return await response.Content.ReadFromJsonAsync<List<Supplier>>() ?? new();
    }

    public static async Task<Supplier> CreateSupplierAsync(Supplier supplier)
    {
        var response = await http.PostAsJsonAsync("api/suppliers", supplier);
        await EnsureSuccessAsync(response);
        return (await response.Content.ReadFromJsonAsync<Supplier>())!;
    }
```

Step 9 – The page

Create SuppliersPage exactly as in Section 12 (drag the five controls, double-click the button) and paste the code from there.

Step 10 – Add it to the menu

client/AppClient/Forms/MainForm.cs

```
new("Products", () => new ProductsPage()),
new("Suppliers", () => new SuppliersPage()),      // ← new
```

CHECKPOINT Run run-dev.bat, open Suppliers, type a name and phone, click Add supplier: the list grows and the row is in PostgreSQL. You built a full-stack feature!

Checklist for every new feature

- [ ] Migration file → npm run db:migrate
- [ ] Model with all the SQL ($1, $2 placeholders)
- [ ] Controller: validate input, choose status codes
- [ ] Routes file + one app.use line in app.js
- [ ] Tested with the browser / Postman (success and error cases)
- [ ] C# model with [JsonPropertyName] for snake_case columns
- [ ] ApiClient methods (Get…Async, Create…Async …)
- [ ] Page (UserControl) + one menu line
- [ ] Tried in the app, including what happens when the server is stopped
- [ ] Committed on a feature branch, pull request opened

SECTION 18

How To: Debug

Bugs are normal. The skill is finding where the data stops being what you expect.

C# in Visual Studio

Action	How
Pause on a line	Click the grey margin (or F9) to set a red breakpoint, then F5
Step through	F10 next line, F11 step into a method, F5 continue
Look at values	Hover a variable, or use the Locals / Watch windows
See the exception	Visual Studio stops on unhandled exceptions – read the message and the line
Print a value	`System.Diagnostics.Debug.WriteLine($"price={price}");` (Output window)

Node.js server

Action	How
Print a value	`console.log('body:', req.body);` – shows in the server window
Errors	errorHandler prints the full error with console.error
Test one endpoint	Browser for GET, Postman / Bruno for POST, PUT, DELETE
On Render	Dashboard → your service → Logs

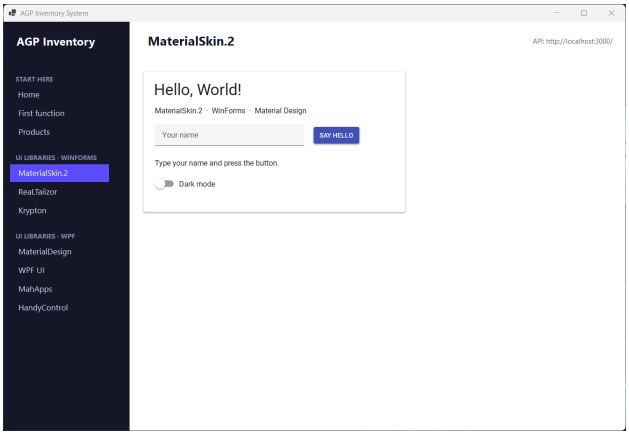
Find which part is broken

1. Does **/api/health** answer? No → the server is not running (look at its window).
2. Does **/api/health/db** answer? No → .env, ca.pem or Aiven is the problem.
3. Does the endpoint work in the browser/Postman? No → the bug is in the server.
4. Yes → the bug is in the C# page, model or ApiClient. Set a breakpoint after the ApiClient call.

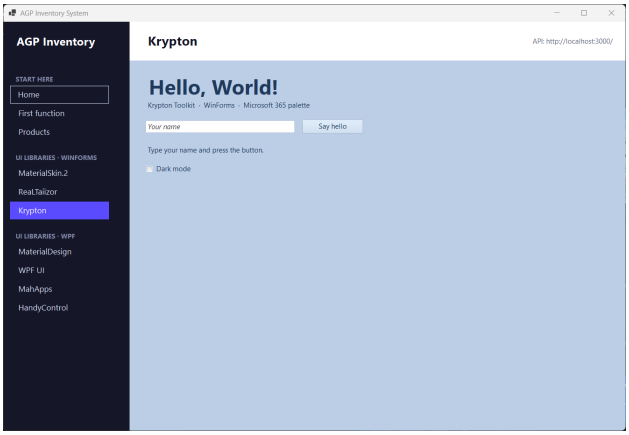
SECTION 19

UI Libraries for C#

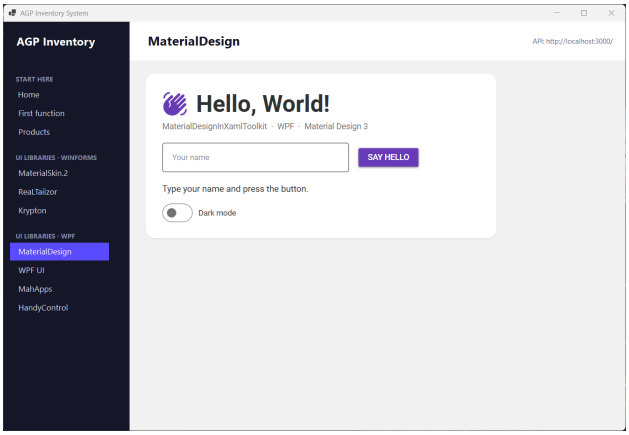
Every library below has a page in the app (menu → UI LIBRARIES) with a TRY THIS list at the top of its code. Pick one and use it for all your pages.



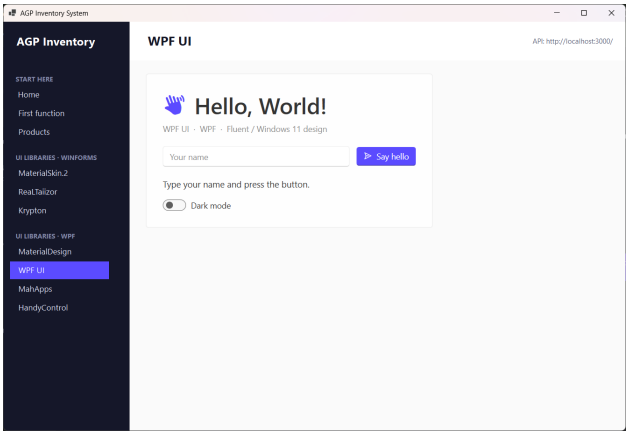
MaterialSkin.2 (WinForms)



Krypton Toolkit (WinForms)



MaterialDesignInXaml (WPF)



WPF UI (WPF)

Windows Forms

Library	NuGet package	Licence	Page in the app
MaterialSkin.2	`MaterialSkin.2`	MIT – free	`Pages/Libraries/MaterialSkinPage.cs`
ReaLTaiizor	`ReaLTaiizor`	MIT – free	`Pages/Libraries/ReaLTaiizorPage.cs`
Krypton Toolkit	`Krypton.Toolkit`	BSD-3 – free	`Pages/Libraries/KryptonPage.cs`
Guna UI 2 / Siticone / Bunifu	–	Commercial (trial)	not included
Syncfusion / DevExpress / Telerik	–	Community licence / commercial	not included

WPF (shown inside the app through AppClient.WpfPages)

Library	NuGet package	Licence	Page in the app
MaterialDesignInXaml	`MaterialDesignThemes`	MIT – free	`MaterialDesignPage.xaml`
WPF UI	`WPF-UI`	MIT – free	`WpfUiPage.xaml`
MahApps.Metro	`MahApps.Metro`	MIT – free	`MahAppsPage.xaml`
HandyControl	`HandyControl`	MIT – free	`HandyControlPage.xaml`

How to use the library you chose

- **WinForms library**: copy the controls from its demo page into your pages (e.g. MaterialButton instead of Button). For MaterialSkin, you can make MainForm inherit MaterialForm for a fully Material window.
- **WPF library**: build your pages as WPF UserControls in AppClient.WpfPages (copy the Resources block from the demo page) and show them with WpfHostPage, or move to a full WPF app later.
- Keep the starter pages working as you restyle them – change one page at a time and run the app after each change.

TIP Pick ONE library. Mixing several theme libraries on the same pages gives controls that look and behave differently. The full comparison, with online template screenshots, is in docs/UI-LIBRARIES.md.

SECTION 20

Working as a Team with Git and GitHub

Git lets several people change the same project without overwriting each other's work.

Terminal

```
git checkout main
git pull                                  # 1. get everyone's latest work
git checkout -b feature/suppliers         # 2. a branch for YOUR task

# ... change code, run-dev.bat, test ...

git status                                # 3. check what changed (no .env!)
git add .
git commit -m "Add suppliers page"        # 4. save a snapshot
git push -u origin feature/suppliers      # 5. upload, then open a Pull Request
```

Type	Branch	Commit message
Feature	`feature/suppliers`	Add suppliers page and API
Bug fix	`fix/product-delete-404`	Fix 404 when deleting a product
Docs	`docs/api-endpoints`	Document supplier endpoints

- Pull every morning; keep branches short (merge within a few days).
- Only one person edits a page's `.Designer.cs` at a time – designer files conflict badly. Assign each page to one owner.
- Several people adding menu lines in MainForm.cs at once is fine – Git merges different lines.
- In a conflict, keep the right lines, delete the `<<<<<<<` / `=======` / `>>>>>>>` markers, build, then commit.

SECTION 21

Security Basics

Even a school project should follow these – they are also what reviewers look for.

- [ ] **No secrets in Git.** `.env` and `ca.pem` are already in .gitignore. Check `git status` before each commit.
- [ ] **No secrets in the C# app.** The .exe only knows the API address.
- [ ] **Placeholders only.** `pool.query(sql, [values])` – never string concatenation.
- [ ] **Validate on the server** even if the app already checks – anyone can call the API with Postman.
- [ ] **Safe error messages.** Log details on the server, send a short message to the user (errorHandler does this).
- [ ] **Hash passwords** with `bcrypt` when you add users.
- [ ] **Protect private endpoints** with a login token (Section 22).
- [ ] **Leaked secret?** Reset the Aiven password at once and update .env and Render.

SECTION 22

Next Step: Adding Login (bcrypt + JWT)

An outline to follow once your first features work. Build it like any feature: migration, model, controller, routes, page.

Terminal

```
cd server
npm install bcrypt jsonwebtoken
```

Users table

```
-- database/migrations/00X_create_users_table.sql
CREATE TABLE users (
    id             SERIAL       PRIMARY KEY,
    username       VARCHAR(50)  NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,      -- never the real password
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
```

1. **Register:** `const hash = await bcrypt.hash(password, 10);` then INSERT username + hash.
2. **Login:** find the user, `await bcrypt.compare(password, user.password_hash)`; if true, `jwt.sign({ id: user.id }, process.env.JWT_SECRET, { expiresIn: "1d" })` and return the token.
3. **Protect routes:** middleware reads `Authorization: Bearer <token>`, checks it with `jwt.verify`, returns 401 if invalid. Use it like `app.use("/api/products", requireAuth, productRoutes)`.
4. **C#:** a LoginPage calls the login endpoint, then `http.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token);` in ApiClient.

SECTION 23

Troubleshooting

Read the error message carefully – it usually says exactly what is wrong.

Problem / message	Likely cause	Fix
`'node' / 'dotnet' is not recognized`	Not installed, or terminal opened before installing	Install it, close and reopen the terminal.
Home: "Server: not reachable"	Server not running	`run-dev.bat server` and read its window.
Home: "Database: not reachable"	.env, ca.pem or Aiven problem	Open `localhost:3000/api/health/db` and read the message in the server window.
`DATABASE_URL is missing`	No server/.env	Copy .env.example to .env and fill it in.
`CA certificate not found`	ca.pem missing / wrong DB_CA_PATH	Save it as server/certs/ca.pem (Render: secret file ca.pem).
`self-signed certificate in certificate chain`	`?sslmode=require` still in DATABASE_URL	Remove it from the end of the URL.
`relation "products" does not exist`	Migrations not run on this database	`npm run db:migrate`
`EADDRINUSE :::3000`	Server already running	Close the other server window.
Render: "Cannot find package.json"	Root Directory not set	Set Root Directory to `server`.
App waits a long time, then fails	Render free service waking up, or wrong URL	Wait / check AppConfig URL.
Grid columns empty or 0	JSON name ≠ property name	Add `[JsonPropertyName("column_name")]`.
Window freezes	`.Result` / `.Wait()` on an async call	Use async / await all the way.
New page not in the menu	Menu line missing	Add it to the menu list in MainForm.cs.
Designer shows an error	Code edited inside .Designer.cs	Undo that edit (git checkout the file).

SECTION 24

Final Checklist

Go through this before you submit or present.

Database

- [ ] All migrations are numbered and have run on Aiven.
- [ ] Seed data exists for the demo.
- [ ] ERD image is in database/diagrams/.

Server

- [ ] Every endpoint tested (success and error cases).
- [ ] No secrets in code; .env not in Git.
- [ ] All SQL uses $1, $2 placeholders.
- [ ] Live on Render; /api/health/db answers ok.

App

- [ ] AppConfig points to the Render URL.
- [ ] One UI library used consistently on every page.
- [ ] Every button shows loading / success / error feedback.
- [ ] Nothing crashes when the server is stopped (try it!).
- [ ] Unused demo pages removed from the menu.

Project

- [ ] READMEs updated (how to run, endpoints).
- [ ] All branches merged; main works from a fresh clone with run-dev.bat setup.
- [ ] Demo rehearsed; Render warmed up 5 minutes before presenting.
- [ ] Release build made (Build → Publish).

SECTION 25

Glossary and Useful Links

Term	Meaning
async / await	Keywords to wait for slow work (network, database) without freezing.
CA certificate	Proves the database server really is Aiven's, so the connection cannot be intercepted.
Controller	Server function that handles one request and chooses the response.
ElementHost	A WinForms control that shows WPF content (used for the WPF library pages).
Middleware	Express function that runs before or after routes (JSON parsing, errors, auth).
Migration	A numbered SQL file that changes the database structure.
Model	Server file with the SQL for one table – or, in C#, a class that holds data.
NuGet / npm	Package managers for .NET / Node.js.
Page	A UserControl shown inside the main window.
Pool	A set of reusable database connections.
Pure function	A function that only uses its inputs and returns a result (easy to test).
Seed	Sample data inserted for testing.
UserControl	A reusable piece of screen designed in Visual Studio.

Topic	Link
Express	expressjs.com
node-postgres (pg)	node-postgres.com
PostgreSQL tutorial	postgresql.org/docs/current/tutorial.html
Render – Node/Express	render.com/docs/deploy-node-express-app
Aiven for PostgreSQL	aiven.io/docs/products/postgresql/get-started
C# for beginners	learn.microsoft.com/dotnet/csharp
Windows Forms	learn.microsoft.com/dotnet/desktop/winforms
HttpClient & JSON	learn.microsoft.com/dotnet/fundamentals/networking/http/httpclient
Git book	git-scm.com/book